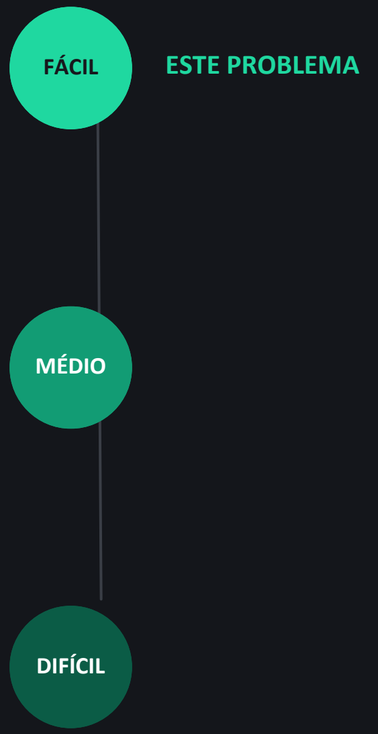
TÓPICOS EM ALGORITMOS E ESTRUTURAS DE DADOS

Parênteses Balanceados



- Alan Andrade Vasconi de Souza
- Ana Livia Turazzi
- Pedro Cândido Sálvio
- Matheus Henrique Souza Carvalho

Desafio de Processo Seletivo

O QUE O DESAFIO PEDE

Parênteses Balanceados

O ENUNCIADO, EM RESUMO

- Dada uma string contendo apenas os caracteres ( ) { } [ ], determinar se ela é válida.
- Uma string é válida quando todo símbolo de abertura tem um fechamento do mesmo tipo, na ordem correta.
- Aberturas devem ser fechadas na ordem inversa em que apareceram — "([)]" é inválida, mas "([])" é válida.
- **Restrição-chave: não basta contar quantos símbolos de cada tipo existem — a ordem de fechamento é o que realmente complica o problema.**

PERGUNTADO POR

- Amazon
- Microsoft
- Bloomberg
- Facebook

NÍVEL DE DIFICULDADE

Fácil na Contagem, Traiçoeira na Ordem

FÁCIL

Classificação oficial do problema no LeetCode

A dificuldade "Fácil" esconde uma pegadinha comum: candidatos tentam resolver só contando símbolos, e travam nos casos de ordem errada.

- Parênteses Balanceados é um dos primeiros problemas de pilha que aparecem em entrevistas.
- É bastante usado pois demonstra se o candidato entende a diferença entre "ter os símbolos certos" e "ter os símbolos na ordem certa" — e se reconhece que esse é exatamente o comportamento de uma pilha (LIFO).
- **O nível "Fácil" mede se você sabe usar uma pilha — a entrevista mede se você sabe por que ela é a estrutura certa aqui.**

CONCEITOS DE ESTRUTURA DE DADOS

Por que a Stack resolve?

String (entrada do problema)

- É só uma sequência de caracteres — não guarda relação entre abertura e fechamento.
- Contar aberturas e fechamentos separadamente não garante que a ordem esteja certa.
- Sem memória do "último símbolo aberto", não dá pra saber qual deveria fechar agora.

Pilha / Stack (solução)

- Sempre expõe o último elemento inserido — exatamente o símbolo de abertura mais recente.
- Empilhar (push) numa abertura e desempilhar (pop) num fechamento custa O(1) por operação.
- Se o topo da pilha não combina com o fechamento atual, a string já está inválida.

A pilha "lembra" a ordem que a string esqueceu: **LIFO — o último a entrar é o primeiro a sair —** e é essa regra que valida a string.

SOLUÇÃO MAIS COMUM

Substituição repetida na String

$$O(n^2)$$

A string é reescaneada a cada rodada de remoção

É a ideia mais intuitiva pra quem ainda não pensou em pilha — funciona, mas reprocessa a string inteira várias vezes.

COMO FUNCIONA

1. Procuramos por pares adjacentes válidos na string, como "()", "[]" ou "{}".
2. Sempre que encontramos um par adjacente, removemos os dois caracteres da string.
3. Repetimos a varredura, porque remover um par pode criar um novo par adjacente (ex.: "(())" vira "()" depois de uma rodada).
4. Se ao final a string ficar vazia, ela era balanceada; se sobrar algo, era inválida.

Substituição repetida na string

```python
def isValid(self, s: str) -> bool:
    pares = ("()", "[]", "{}")

    tamanho_anterior = -1
    while tamanho_anterior != len(s):
        tamanho_anterior = len(s)
        for par in pares:
            s = s.replace(par, "")

    return len(s) == 0
```

SOLUÇÃO COM MELHOR COMPLEXIDADE

Pilha: Uma única passagem

Cada abertura empilhada é uma "promessa" que a pilha cobra na ordem certa.

O(n)

A string é percorrida uma única vez

É a solução ideal já que soluciona o problema com uma única leitura da string, atendando a símbolos certos e na ordem certa

COMO FUNCIONA

1. Percorremos a string caractere por caractere, uma única vez.
2. Se o caractere é uma abertura (([{), empilhamos (push).
3. Se é um fechamento, olhamos o topo da pilha: se combina com o tipo certo, desempilhamos (pop); se não combina — ou a pilha está vazia — a string já é inválida.
4. Ao terminar a string, ela só é válida se a pilha ficar vazia, sem aberturas pendentes.

Solução usando Pilha/Stack

```python
def isValid(self, s: str) -> bool:
        pilha = []
        fechamento_para_abertura = {
            ')': '(',
            ']': '[',
            '}': '{',
        }

        for caractere in s:
            if caractere in fechamento_para_abertura:
                # É um fechamento: verifica se o topo da pilha combina
                if pilha and pilha[-1] == fechamento_para_abertura[caractere]:
                    pilha.pop()
                else:
                    return False
            else:
                # É uma abertura: empilha
                pilha.append(caractere)

        # Só é válida se não sobrou nenhuma abertura pendente
        return len(pilha) == 0
```

ANÁLISE DE COMPLEXIDADE

Complexidade: O Ganho de Usar uma Pilha

Operação	Substituição Repetida	Pilha (Stack)
Tempo (pior caso)	$O(n^2)$	**O(n)**
Espaço extra	O(n) (cópias da string)	O(n) (pilha)
Passagens pela string	várias (até n/2)	**1**

Essa comparação entre reprocessar a string várias vezes e resolvê-la em uma única passagem é exatamente a análise de complexidade de tempo e espaço em notação Big-O: escolher a estrutura certa evita trabalho repetido.